Adevar Labs

Prepared for:
Trepa

November 10, 2025

Trepa Solana Program Audit Report

Version: after-fix report

Table of Contents

1. Executive Summary

About Trepa

Trepa is a decentralized prediction platform for continuous outcomes built on Solana. It runs on score-weighted, time-adjusted, ROI-capped pari-mutuel pools, where all user stakes from a prediction round are aggregated into a single shared pool and then redistributed algorithmically based on each participant's relative prediction accuracy, verified via Merkle proofs. Unlike traditional prediction markets, Trepa doesn't rely on counterparties or order books. Instead, it provides infinite buy-in liquidity and zero counterparty risk, since users collectively contribute to and earn from the same pool.

Audit Summary

All issues have been fixed.

Number of Findings: 4 total

- Critical: 0

- High: 1

- Medium: 0

- Low: 3

Number of Enhancement Opportunities: 10

A study on the Merkle Tree implementation has also been conducted and can be found on page 34.

Risk Profile

The following table summarizes the distribution of identified vulnerabilities by risk level:

Risk Level	Count	Fixed	Acknowledged
Critical	0	–	–
High	1	1	0
Medium	0	–	–
Low	3	3	0

Key Findings Highlight

- (Fixed) Vulnerability to log truncation in event emissions may cause incomplete event parsing and inaccurate reward distributions
- (Fixed) Using non-canonical SPL token accounts enables stake distribution across multiple pools, causing consolidation difficulties
- (Fixed) Unclaimed rewards are not handled correctly, leading to locked prediction accounts and lost rent refunds

Overall Security Posture

The following sections describe the security posture before and after the fix review.

The fix review is the stage of the audit where Adevar Labs checked the fixes implemented by the Trepa team for the issues found during the audit.

After-fix review

The Trepa team responded promptly to the audit findings and successfully implemented all recommended fixes. By migrating to **emit_cpi!** for event emissions, the team eliminated the risk of log truncation that could have compromised reward distribution integrity.

All three low-severity issues were also addressed, which include the implementation of an on-chain **max_roi** limit and non modifiable **platform_fee**.

Beyond security fixes, the team addressed all 10 enhancement opportunities, further strengthening the codebase's maintainability.

The systematic resolution of all identified issues demonstrates the team's commitment to security and positions Trepa for a secure mainnet deployment.

Before-fix review

The Trepa team has demonstrated solid engineering practices in building a functional prediction platform.

The protocol features a clean architecture and development practice, showing a good understanding of the Solana environment and of the Anchor framework.

During the initial assessment we noted that the protocol relied on off-chain systems and trusted actors for correct reward computation, creating a significant trust assumption.

To mitigate this risk, the protocol should include additional on-chain validation for elements that directly impact reward distribution.

Specifically, it should verify that the **max_roi** limit is respected and that the pre-defined **platform_fee** is correctly applied.

Launch Recommendations

After-fix review

All mandatory and strongly recommended items from the "Before-fix Review" section have been successfully addressed.

As the protocol matures, consider progressive decentralization through multi-resolver consensus or on-chain outcome oracles. The current implementation provides a solid foundation for these evolutionary improvements.

Before-fix review

I. Mandatory before launch:

- Fix the "High" severity issues:
- Migrate to **emit_cpi!** to resolve log truncation and prevent information loss that could corrupt reward calculations.

II. Strongly Recommended:

- Address the non-canonical pool token account vulnerability.
- Implement proper unclaimed reward handling to enable protocol rent reclamation.
- Resolve the **platform_fee** vs **protocol_fee** inconsistency to ensure transparent fee collection.
- Implement on-chain validation to reduce blind trust in off-chain computation.

III. Post-Launch Monitoring:

- Establish clear procedures for private key security management and post-hack management.
- Monitor event emission and implement alerting for log issues that could corrupt resolver data.
- Implement monitoring dashboards showing resolver decisions, fee calculations, and reward distributions for community review.
- Track unclaimed reward accumulation and consider implementing automated sweeping mechanisms.
- Track statistical anomalies.

IV. Future Considerations:

- Consider publishing resolver logic, implementing verifiable computation proofs, or establishing a multi-resolver consensus mechanism.
- Explore progressive decentralization such as on-chain outcome oracles or multi-resolver consensus.
- If transitioning to permissionless pool creation, implement on-chain token allowlists and extension validation.

Audit Scope

- **Repository:** https://github.com/TrepaOrg/trepa-onchain-svm
- **Commit Hash:** **26b4fbf0caf8607fa8d4ff036cf0a355c8e1f180**
- **Files/Modules in Scope:**
- programs/trepa/src/*.rs
- utils/**/*.ts

2. Assumptions and Considerations

Audit Assumptions

I. The protocol will only utilize vetted and trusted tokens (such as USDC, USDT, and other battle-tested tokens) for staking.

II. The blockchain only stores the stake value, not the prediction value itself. Prediction values are emitted as events and indexed off-chain. The accuracy and integrity of off-chain indexing systems are assumed to function correctly.

Centralization Risks

I. Resolver and creator addresses are trusted entities within the system. The audit assumes these roles will act honestly and in accordance with protocol rules.

Trust Assumptions

I. The protocol relies on off-chain infrastructures for event indexing and processing to function.

II. The protocol provides no on-chain mechanism (oracles) to verify that outcomes are correctly determined and users must trust Trepa for providing correct outcomes.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Impact

Impact reflects the potential consequences of the issue–particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

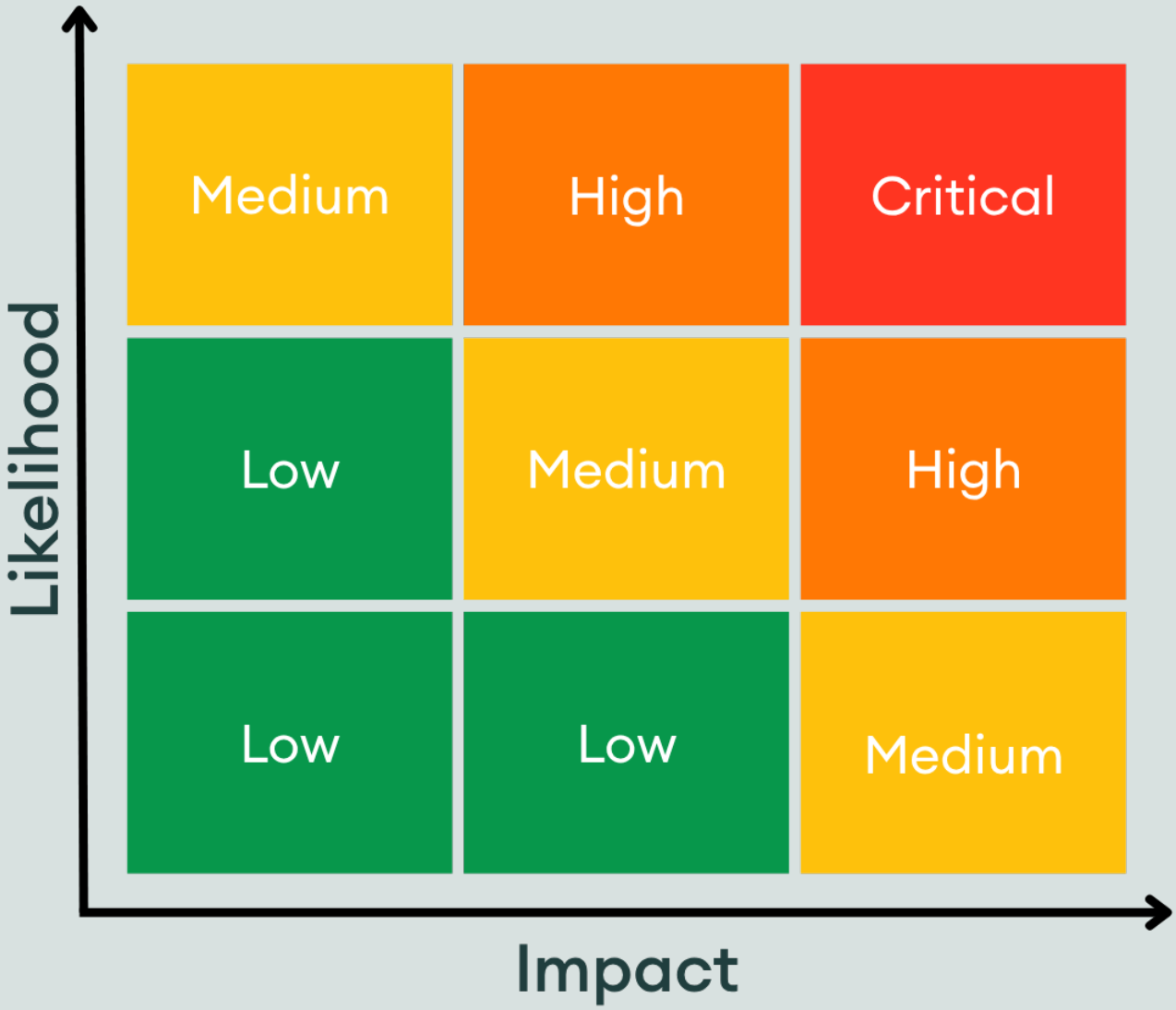
- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substansial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability
- **Low**: Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

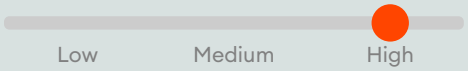
4. Findings

H01: Vulnerability to Log Truncation in Event Emissions

Status: Resolved

Impact:

Likelihood:



Severity: High

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L83

>_ lib.rs RUST

```
81:             new_status
82:         );
83:         emit!(ProgramStatusToggledEvent {
84:             config_account: config.key(),
85:             admin: config.admin,
```

Description:

The audited system extensively uses Anchor's **emit!** macro to log events for every instruction, with the off-chain backend service relying on these logs for critical computations like settlement and reward distribution. However, Solana's logging syscalls (**sol_log_data**) are prone to truncation by RPC providers (logs are truncated if it gets longer than 10kB).

A malicious user could force log-truncation by including a "spam-log" IX as the first IX in a TX. Any programs called later in the same TX have no way of detecting the log truncation, and all events will be silently dropped.

This can result in partial or missing event data, leading to:

- Incomplete parsing of event payloads (e.g., serialized Borsh data for rewards or settlements).
- Inaccurate off-chain calculations, such as under/over-allocation of rewards.
- Potential user losses from erroneous distributions, disputes, or compliance issues.
- Increased failure rates in high-volume transactions where logs accumulate quickly.

This pattern amplifies risks in production, especially on public RPCs, and violates best practices for reliable event emission.

Recommendation:

Migrate all **emit!** calls to Anchor's CPI-based events (**emit_cpi!**) to embed full payloads in transaction **innerInstructions**.

Steps:
1. Enable Feature: Add **event-cpi = ["anchor-lang/event-cpi"]** to **Cargo.toml** features.
2. Annotate Events: Tag event structs with **#[event_cpi]** to generate noop CPI handlers.
3. Update Code: Replace **emit!(Event { ... })** with **emit_cpi!(Event { ... })** in all instructions;
IMPORTANT: ensure auth (e.g., admin signers) in event accounts to prevent event spoofing.

4. Indexer Adaptation: Modify the backend to parse **getTransaction** → **meta.innerInstructions** → decode ix data via Anchor's coder (e.g., **@coral-xyz/anchor** in JS/TS). Use enhanced RPCs (Helius/Triton) for real-time polling.
5. Testing: Simulate high-log txs on devnet; audit for full payload recovery.

This fix ensures data integrity without refactoring core logic, reducing user risk and operational errors.

Proof of Concept:

Deploy to devnet; simulate a **finalize_pool** + multi-**claim_rewards** tx on public RPC. Fetch logs via **getTransaction**, assert truncation on **emit!** (partial base64), full recovery on **emit_cpi!**.

```typescript
const txSig = await program.methods.finalizePool(merkleRoot, protocolFee, claims).rpc();
const tx = await connection.getTransaction(txSig, { commitment: 'confirmed' });
const logs = tx.meta?.logMessages || [];  // Truncated on emit!
expect(logs.slice(-1)[0]).to.match(/truncated/);  // Fails parsing
```

Developer Response:

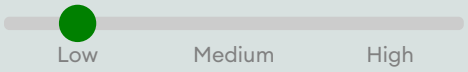
Fixed according to our recommendations: PR #47

L01: Unclaimed rewards are not handled correctly

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L568

>_ lib.rs RUST

```
566:
567:     /// Claim the rewards for a prediction
568:     pub fn claim_rewards(
569:         ctx: Context<ClaimRewards>,
570:         amount: u64,
```

Description:

The function **claim_rewards** is used to claim the rewards from the prediction. However, there might be cases where some users don't claim their rewards from the pool after the pool is finalized. Once the pool is finalized and the root is set, users can start claiming. However, the onchain logic has no end time for this, and there could be cases where some winners don't claim those winning rewards. These unclaimed rewards need to be handled correctly onchain.

The result of this will be:
• The protocol cannot close the prediction account and get the rent refunded which it paid for.
• The likelihood of such an event increases for very small rewards considered not worth being claimed by the users.

Recommendation:

Adding a time period to claim the rewards would be a better approach so that unclaimed rewards can be claimed by the bot with off-chain logic.

Developer Response:

Fixed according to our recommendations: PR #43

L02: Missing Validation Constraints for Prediction Outcome Bounds and Step Size in Pool Creation

Status: Resolved

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L207-L209

>_ lib.rs RUST

```
205:         );
206:
207:         require!(min_outcome < max_outcome, CustomError::InvalidOutcomeBounds);
208:
209:         require!(step > 0, CustomError::InvalidStepValue);
210:
211:         require!(max_roi > 0, CustomError::InvalidMaxRoi);
```

Description:

The **create_pool** function validates basic bounds (e.g., **min_outcome < max_outcome**, **step > 0**) but omits constraints ensuring the prediction range is usable and fully coverable by valid steps. Specifically:

• Missing Constraint 1: **(max_outcome - min_outcome) >= step**. Without this, if the range is smaller than the step (e.g., **min_outcome=0**, **max_outcome=1**, **step=2**), no valid predictions are possible, rendering the pool immediately unusable.

• Missing Constraint 2: **(max_outcome - min_outcome) % step == 0**. This ensures the range is an exact multiple of the step, preventing gaps at the upper bound. For example, with **min_outcome=4**, **max_outcome=9**, **step=2**, users cannot predict 9 (since **9 % 2 == 1**), leaving the max outcome uncovered.

• Missing Constraint 3: **min_outcome % step == 0**. This aligns the lower bound to a valid prediction value. For example, with **min_outcome=3**, **max_outcome=9**, **step=2**, neither 3 nor 9 is predictable (**3 % 2 == 1**, **9 % 2 == 1**), excluding both endpoints and creating incomplete coverage.

These gaps allow creation of "broken" pools that pass initialization but fail during **predict/update_prediction** (e.g., **CustomError::PredictionOutOfBounds**), leading to:

• User frustration: Attempts to predict result in errors, potentially causing churn (e.g., users abandon Trepa for competitors).
• Economic waste: Rent-exempt pool accounts (~0.02 SOL) are created but discarded as unusable, inflating costs for creators.
• Griefing vector: Malicious actors could spam invalid pools to clutter the platform or DoS legitimate usage via indexer overload.
• Subtle UX issues: Even valid ranges might exclude intended outcomes, undermining the social predictions model's accuracy and trust.

While not a direct fund loss, this erodes platform adoption in a competitive DeFi space, especially for time-sensitive pools.

Recommendation:

Either add the three constraints immediately after existing validations in **create_pool** to enforce usable ranges or restrict **step = 1**.

Developer Response:

Trepa completely removed the step parameter from the pool account structure: PR #34

L03: Using non-canonical SPL token account can distribute stake across many prediction pools

Status: Resolved

Impact:

Likelihood:

Severity: Low

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/context.rs#L154-L155

>_ context.rs RUST

```
152:     #[account(
153:         mut,
154:         constraint = pool_token_account.owner == pool.key() @ CustomError::InvalidTokenAccountOwner,
155:         constraint = pool_token_account.mint == stake_token_mint.key() @ CustomError::InvalidMint
156:     )]
157:     pub pool_token_account: Account<'info, TokenAccount>,
```

Description:

The **Predict** function context for **pool_token_account** only checks that the **owner** and **mint** are matching the prediction pool. It does not verify that the SPL token account is the canonical ATA created by the associated token account program. As a result, an attacker can create many SPL token accounts and call the **predict** function to distribute their stake across these. The effects of this are:
• Consolidation of staked funds is difficult due to being distributed across many accounts
• Event confusion due to each emitted **prediction** having multiple token accounts

See similar issue

Recommendation:

In **predict** and all other functions using the **pool_token_account**:
Ensure the account is derived as the ATA by adding a constraint for ATA derivation within the function context. For example:

RUST

```
pool_token_account.key() == get_associated_token_address(&pool.key(), &stake_token_mint.key())
```

Alternatively, leverage the **associated_token_program** anchor constraints. For example:

RUST

```
#[account(
        mut,
        associated_token::mint = stake_token_mint,
        associated_token::authority = pool,
```

... continued RUST

```
    )]
    pub pool_token_account: Account<'info, TokenAccount>,
...
pub associated_token_program: Program<'info, AssociatedToken>,
```

Proof of Concept:

A malicious user can create many SPL token accounts and make small predictions in each account separately by passing the different accounts to the **predict** function context.

Developer Response:

Fixed according to our recommendations: PR #35

5. Enhancement Opportunities

E01: Usage of Magic Numbers reduces code maintainability

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L334

>_ lib.rs RUST

```
332:                     .checked_mul(parlay_account.fee_percentage)
333:                     .ok_or(CustomError::ArithmeticError)?
334:                     .checked_div(10000)
335:                     .ok_or(CustomError::ArithmeticError)?;
336:
```

Description:

The codebase uses the hardcoded value 10000 as a basis point divisor in multiple locations throughout **lib.rs** without defining it as a named constant. Basis points are a standard financial unit where 10,000 basis points = 100%, but the repeated use of this magic number reduces code maintainability, readability, and increases the risk of errors.

Multiple places where usage of Magic number occurs are as follow :

1. https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L47

2. https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L134

3. https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L334

4. https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L701

This is primarily a code quality issue, potential consequence for this is If the basis point standard needs to change (e.g., for higher precision), all hardcoded instances must be manually located and updated, increasing the risk of missing one.

Potential Benefit:

- The code will be easier to maintain and update
- Reduced risk of typos or inconsistent values

Recommendation:

In the lib.rs add following lines

RUST

```
// Constants
pub const BPS_IN_100_PERCENT: u64 = 10000; // 100%


// Line 47 - initialize()
require!(platform_fee <= BPS_IN_100_PERCENT, CustomError::InvalidPercentage);
```

... continued RUST

```rust
// Line 134 - update_config()
require!(platform_fee <= BPS_IN_100_PERCENT, CustomError::InvalidPercentage);

// Lines 331-335 - finalize_pool()
let streak_fee = protocol_fee
    .checked_mul(streak_account.fee_percentage)
    .ok_or(CustomError::ArithmeticError)?
    .checked_div(BPS_IN_100_PERCENT)
    .ok_or(CustomError::ArithmeticError)?;

// Line 703 - register_streak()
require!(fee_percentage <= BPS_IN_100_PERCENT, CustomError::InvalidPercentage);
```

Update error messages in errors.rs:

RUST

```rust
#[error_code]
pub enum CustomError {
    #[msg("The percentage must be between 0 and BPS_IN_100_PERCENT (10000 = 100%)")]
    InvalidPercentage,
}
```

Developer Response:

Fixed according to our recommendations: PR #39

E02: Duplicate Merkle tree construction implementation creates maintainability risk

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/utils/merkleTree/createMerkleTree.ts#L40-L58

>_ createMerkleTree.ts TYPESCRIPT

```
38:   }
39:
40:   function computeMerkleRoot(nodes: Buffer[]): Buffer {
41:     let currentLevel = nodes;
42:     while (currentLevel.length > 1) {
43:       const nextLevel: Buffer[] = [];
44:       for (let i = 0; i < currentLevel.length; i += 2) {
45:         if (i + 1 < currentLevel.length) {
46:           // Sort the two sibling hashes lexicographically.
47:           const sortedPair = [currentLevel[i], currentLevel[i + 1]].sort(
48:             Buffer.compare,
49:           );
50:           nextLevel.push(hashPair(sortedPair[0], sortedPair[1]));
51:         } else {
52:           // Duplicate the last node if there is an odd number.
53:           nextLevel.push(hashPair(currentLevel[i], currentLevel[i]));
54:         }
55:       }
56:       currentLevel = nextLevel;
57:     }
58:     return currentLevel[0];
59:   }
60:
```

Description:

The Merkle tree construction logic is duplicated across two functions with slightly different implementation, even though end result is the same.
The only remarkable difference being that in **createMerkleTree.ts**, odd nodes are handled inline during the pairing loop with conditional checks. In **getMerkleProof.ts**, the array is pre-padded by duplicating the last node before the pairing loop begins.
While both approaches produce the same result, this inconsistency creates maintenance burden increasing the risk of introducing bugs during future modifications.

Potential Benefit:

- Reduced code duplication.
- Improved maintainability: Single source of truth for tree construction ensures consistency.
- Lower bug risk: Future modifications only need to be made in one place, preventing divergence.
- Better testability: Shared logic can be unit tested independently.

Recommendation:

Extract the Merkle tree building logic into a shared utility function that returns all tree levels. This allows both functions to use identical tree construction while serving their different purposes.

Developer Response:

Fixed according to our recommendations: PR #38
The Trepa team also strengthened the merkle tree implementation by adding an explicit leaf/node domain separation. See Appendix A for more details.

E03: Arbitrary fee is passed during finalize pool

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L281

>_ lib.rs RUST

```
279:         ctx: Context<'key, 'accounts, 'remaining, 'info, FinalizePool<'info>>,
280:         merkle_root: [u8; 32],
281:         protocol_fee: u64,
282:         claims: u32, // number of claims in the merkle tree
283:     ) -> Result<()> {
```

Description:

platform_fee is initialized and set but never used. Also, **update_config** is called by the admin to change the **platform_fee**, but this **platform_fee** isn't used anywhere in the code.

Instead, in the function **finalize_pool**, resolvers pass an arbitrary **protocol_fee** which is not consistent with the initialized fee in the contract.

Potential Benefit:

- Aligning the fee during the finalization of the pool will increase user trust in the protocol with reduced centralization.
- Meaningful use of **update_config** to update the platform fee.

Recommendation:

Use the initialized fee during the finalization of the pool.

Developer Response:

Fixed according to our recommendations: PR #41

E04: On-Chain Enforcement of max_roi Limit in Reward Claims

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/state.rs#L56

>_ state.rs RUST

```
54:
55:     /// Maximum ROI in basis points (e.g., 10000 = 100%)
56:     pub max_roi: u64,
57:
58:     /// The allowed token to predict in this pool
```

Description:

The **PoolAccount** stores a **max_roi** field (maximum return on investment, in basis points) during pool creation, intended to cap individual user rewards relative to their stake (e.g., **reward <= stake * max_roi / 10000**). However, this value is not validated or enforced anywhere **on-chain**:

• In **claim_rewards**, the claimed amount is directly transferred after Merkle proof verification, without checking if it exceeds the user's **stake * max_roi** (where stake is stored in the associated **PredictionAccount**).
• Off-chain, the resolver reportedly uses **max_roi** to compute fair rewards and generate the Merkle tree (per whitepaper and team assumptions), assuming trusted computation.

This design relies entirely on off-chain integrity, creating a trust assumption on the resolver. If the Merkle tree is malformed (e.g., due to backend bugs, malice, or oracle failures), users could claim rewards exceeding **max_roi** limits, leading to over-payouts from the pool (impacting losers/treasury).

Potential Benefit:

On-chain enforcement adds verifiable guarantees, aligning with Solana best practices for DeFi (e.g., preventing "oracle attacks" via proofs). This is especially relevant for a social predictions platform where transparency drives adoption.

Recommendation:

Enforce **max_roi** in **claim_rewards** by cross-checking the claimed amount against the prediction's **stake * pool.max_roi / 10000**, post-proof verification but pre-transfer. This adds a negligible amount of CUs and a revert if exceeded, without altering Merkle logic. Keep off-chain computation for tree generation, but make on-chain the final gatekeeper.

Developer Response:

Fixed according to our recommendations: PR #36

E05: Implement secure access update functionality

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L162-L166

>_ lib.rs RUST

```
160:         }
161:
162:         if let Some(new_creator) = creator {
163:             config.creator = new_creator;
164:         }
165:         if let Some(new_resolver) = resolver {
166:             config.resolver = new_resolver;
167:         }
168:
```

Description:

The protocol implements the **update_access** function which enables the **admin** to modify the Pubkeys associated with **creator**, **resolver** without any additional safeguards. See similar issues 1, 2.

The primary risk here is incorrect address assignment rather than authority compromise. Since only the **creator** and **resolver** addresses are updated (not the **admin** itself), an erroneous update can be corrected by the admin calling **update_access** again with the correct addresses.

Potential Benefit:

Implementing a two-step update process with explicit acceptance by the nominated addresses provides an additional layer of safety by ensuring that the intended recipients verify they control the private keys for the nominated addresses before the update takes effect.

Recommendation:

Implement a two-step access update flow:

1. Proposal by **admin**:
• Introduce a new instruction,**propose_update_access**, that allows the current admin to nominate the new **creator** and **resolver**.
• Update the **config** struct with **pending_creator** and **pending_resolver** fields
• Set these two fields in the new **propose_update_access** function

2. Acceptance by the nominated addresses:
• Introduce two instructions,**accept_creator_role** and **accept_resolver_role**, where the nominated addresses can explicitly accept the transfer of authority.
• In the **accept_creator_role** instruction, verify that the signer is the **pending_creator** and update the **creator** field in the config account to the **pending_creator** address. Do the same for the **resolver**.
• Clear the pending fields to indicate the completion of the transfer of access.

Developer Response:

Fixed according to our recommendations: PR #42

E06: Incomplete Token 2022 implementation will cause incompatibility with Token Extensions

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/context.rs#L460-L470

>_ context.rs RUST

```
458:         constraint = user_token_account.mint == mint.key() @ CustomError::InvalidMint
459:     )]
460:     pub user_token_account: Account<'info, TokenAccount>,
461:
462:     #[account(
463:         seeds = [b"config"],
464:         bump = config.bump,
465:         constraint = config.status == ConfigStatus::Active @ CustomError::ProgramFroze
         n
466:     )]
467:     pub config: Account<'info, ConfigAccount>,
468:
469:     pub mint: Account<'info, token::Mint>,
470:     pub token_program: Program<'info, Token>,
471:
472:     pub system_program: Program<'info, System>,
```

Description:

The program accepts both SPL Token and Token-2022 programs as shown in the **assert_valid_token_account** utils function:

RS

```
if *token_program.key == spl_token::id() {
        // SPL Token
       // ... Proceed to checks
    } else if *token_program.key == spl_token_2022::id() {
        // SPL Token 2022
       // ... Proceed to checks
    } else {
        return Err(CustomError::IncorrectTokenProgramId.into());
    }
```

However, the implementation is incomplete for Token 2022 compatibility:
1. The **token_program** should use the **TokenInterface** type
2. The **token::transfer** call should be replaced by **token::transfer_checked**, as the former is incompatible with the **TransferHook** and **TransferFees** extensions, but future extensions might also be incompatible with **transfer**
3. The **Mint** and **TokenAccount** fields should use the **InterfaceAccount** type instead of **Account**

See the solana-program Token 2022 integration guide for more details, and this example implementation.

Potential Benefit:

Support Token2022 as expected.

Recommendation:

To make the program compatible:
1. Replace **Account<'info, TokenAccount>** with **InterfaceAccount<'info, TokenAccount>**
2. Replace **Account<'info, token::Mint>** with **InterfaceAccount<'info, Mint>**
3. Replace **Program<'info, Token>** with **Interface<'info, TokenInterface>**
4. Replace **token::transfer** calls with **token::transfer_checked**

Token2022 Considerations

Also, by accepting Token2022 it is important to define a list of allowed extensions, as these can significantly impact the program's functioning:
• **TransferFees** extension: every instance where such tokens are transferred should ensure that fees have been taken into account by measuring the actual received amount, rather than relying on the transfer amount argument.
• **TransferHook** extension: allows a program defined by the mint authority to be called for each transfer. This program receives the involved token accounts and the transferred amount. The extension then can execute logic after each transfer, potentially impacting actual transferred amounts.

Example implementation for accurate balance measurement:

```rs
let before_balance = self.destination_ata.amount;

let ctx_accounts = TransferChecked {
    //transferChecked parameters
};

let ctx = CpiContext::new(self.token_program.to_account_info(), ctx_accounts);
transfer_checked(ctx, amount, ctx.accounts.mint.decimals)?;

self.destination_ata.reload()?; // required in order to update the account in memory

let after_balance = self.destination_ata.amount;
let received = after_balance.checked_sub(before_balance).ok_or(ErrorCode::UnderflowError)?;
```

While tokens can be admin-configured with pre-verification in the current design, once pool creation becomes permissionless, if no token allowlist is implemented, and extension allowlist will need to be implemented directly within the program itself.

Example implementation for on-chain extension verification:

```rs
let extensions = get_mint_extensions(&ctx.account.mint)?;

if !extensions.contains(&ExtensionType::PermanentDelegate) {
    return err!(ExtError::InvalidMint);
}
```

List of extensions and security implications: https://neodyme.io/en/blog/token-2022/

Developer Response:

The Trepa team decided to only integrate Token-2022 that do not implement the **TransferFee** or **TransferHook** extension.
Fixes: PR #37

E07: Adding balance validation will prevent transaction failures and improve user experience

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L447-L456

>_ lib.rs RUST

```rust
        }

        // Transfer stake token from the predictor's token account to the pool's token account
        let cpi_ctx = CpiContext::new(
            ctx.accounts.token_program.to_account_info(),
            Transfer {
                from: ctx.accounts.predictor_token_account.to_account_info(),
                to: ctx.accounts.pool_token_account.to_account_info(),
                authority: ctx.accounts.predictor.to_account_info(),
            },
        );
        token::transfer(cpi_ctx, stake)?;

        msg!(
```

Description:

The **createPrediction** function from the Typescript SDK checks if the predictor's ATA exists before constructing the transaction, but does not verify that the account has sufficient token balance to cover the stake amount.
This leads to transaction failures that could be prevented with a simple pre-flight balance check.

Currently **L48-L58**, the code only checks account existence:

JS

```js
const predictorInfo = await connection.getAccountInfo(predictorTokenAccount);
  if (!predictorInfo) {
    // Create ATA if needed
  }
```

However, it proceeds to construct a transaction that will fail if the account has insufficient balance for the token transfer:
• if the ATA exists but don't have sufficient token balance.
• if the ATA does not exist yet and need to be created, it will have no token balance.

As both instructions are bundled as a single transaction, the ATA creation in the secondary path will not happen.

Potential Benefit:

• Better UX: Users receive clear error messages about insufficient balance before submitting transactions
• Reduced transaction fees: Users avoid paying fees for transactions that will fail
• Faster feedback: Immediate client-side validation instead of waiting for on-chain execution failure

Recommendation:

Separate ATA creation from prediction, or require pre-funded ATAs and add balance validation:

```JS
const predictorInfo = await connection.getAccountInfo(predictorTokenAccount);
if (!predictorInfo) {
  // a solution would be to create the ATA and ask user to fund it before calling predict
    again
}

// Validate balance
const balance = await connection.getTokenAccountBalance(predictorTokenAccount);
if (balance.value.amount < Math.floor(stake * 10 ** decimal).toString()) {
  throw new Error(`Insufficient balance: have ${balance.value.uiAmount}, need ${stake}`);
```

Developer Response:

Fixed according to our recommendations: PR #40

E08: Using environment variables will improve maintainability by avoiding hardcoded public keys in source files

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/main/programs/trepa/src/lib.rs#L45

>_ lib.rs RUST

```
43:         resolver: Pubkey,
44:     ) -> Result<()> {
45:         // TODO: Hardcode admin check for mainnet deployment
46:
47:         require!(platform_fee <= 10000, CustomError::InvalidPercentage);
```

Description:

The **initialize** function includes a TODO for adding an admin check ("Hardcode admin check for mainnet deployment"), indicating team awareness of the need to restrict initialization to an authorized party. Without this check, any user could invoke **initialize** and claim control over the platform, though this is mitigated in practice by controlled deployment environments. To align with best practices, implement the check while supporting flexible testing: Use environment variables for local/dev setups (allowing each developer to use their own keypair without sharing privkeys) and inlining for production deployment. This avoids hardcoding (which exposes keys in source/repos and complicates multi-env testing) and ensures the program remains secure post-deploy.

Potential Benefit:

1. No changes to the code and hence the audit commit in the report after the audit is finished.
2. Facilitates testing without the need to use the actual admin key or to share they private key of the admin with multiple people.

Recommendation:

Add the admin authorization check in **initialize** using a build-time injectable **Pubkey** via environment variables and a deployment script. This enables per-dev testing keypairs (via **.env**) while baking the prod key into the binary on deploy. Minimal changes; inherits from the existing TODO.

1. Update **lib.rs** (Injectable placeholder):

RUST

```
pub fn initialize(
        ctx: Context<Initialize>,
        platform_fee: u64,
        treasury: Pubkey,
        creator: Pubkey,
        resolver: Pubkey,
    ) -> Result<()> {
        // Authorized init check (injected at build/deploy)
        require!(
            ctx.accounts.admin.key() == EXPECTED_ADMIN,
            CustomError::Unauthorized
        );
...
```

2. Deployment Script (e.g., **scripts/deploy_with_env.js** run **pre-anchor deploy**):

```javascript
require('dotenv').config();  // Loads .env (e.g., ADMIN_PUBKEY=YourKeyHere)
const fs = require('fs');
const { PublicKey } = require('@solana/web3.js');

const adminPubkey = process.env.ADMIN_PUBKEY;
if (!adminPubkey) throw new Error('Missing ADMIN_PUBKEY in .env—set for dev/prod');

try {
    new PublicKey(adminPubkey);  // Validate Pubkey
} catch (e) {
    throw new Error('Invalid ADMIN_PUBKEY in .env');
}

// Inline replacement
let code = fs.readFileSync('programs/trepa/src/lib.rs', 'utf8');
const placeholder = 'pubkey!("PLACEHOLDER_ADMIN_PUBKEY_TO_BE_REPLACED")';
const replacement = `pubkey!("${adminPubkey}")`;
code = code.replace(placeholder, replacement);
fs.writeFileSync('programs/trepa/src/lib.rs', code);  // Or use temp build dir

console.log(`Injected admin (truncated): ${adminPubkey.slice(0, 8)}...`);
// Now: anchor deploy --provider.cluster mainnet
```

3. **.env** Setup (Gitignore; per-env variants):

```
# .env.dev (for local testing—each dev uses their own keypair)
ADMIN_PUBKEY=DevTestAdminPubkeyHere

# .env.mainnet (for prod deploy—team's shared prod key)
ADMIN_PUBKEY=ProdAdminPubkeyHere
```

- Testing: **ADMIN_PUBKEY=YourLocalKey anchor test** (overrides dummy; no privkey sharing).
- Deploy: Load **.env.mainnet**, run script → deploy. Verify: Query config PDA's admin field matches prod key.

This approach resolves the TODO securely, supports collaborative testing (unique keypairs per dev), and ensures prod hardening without code changes. For future admin actions (e.g., **update_config**), use the stored **config.admin**.

Developer Response:

Fixed according to our recommendations: PR #44

E09: Removing duplicate check will result in cleaner code

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/58d292f0ae4f2adc859073ccabaab269d9fa1091/programs/trepa/src/lib.rs#L579

>_ lib.rs RUST

```
577:         require!(proof.len() <= 32, CustomError::ProofTooLong); // Reasonable limit
578:         require!(amount > 0, CustomError::ZeroClaimAmount);
579:         require!(
580:             ctx.accounts.pool.root != [0u8; 32],
581:             CustomError::PoolNotFinalized
```

Description:

There is a duplicate check for the merkle root in a pool to be non-zero before rewards can be claimed in **claim_rewards**.

Potential Benefit:

This will reduce execution cost and improve code readability.

Recommendation:

Remove the duplicate check.

Developer Response:

Fixed according to our recommendations: PR #33

E10: Changing platform_fee data type will reduce cost of programs

Location: https://github.com/AdevarLabs/trepa-onchain-svm-20251020/blob/58d292f0ae4f2adc859073ccabaab269d9fa1091/programs/trepa/src/state.rs#L9

>_ state.rs RUST

```
7:
8:     /// Platform fee in basis points
9:     pub platform_fee: u64,
10:
11:    /// Treasury account to receive platform fees
```

Description:

The **platform_fee** is using a **u64** data type to represent the fee the platform collects in basis points. However, since the maximum accepted value of this is **10000**, this data type can be changed to **u16**.

Potential Benefit:

Changing the data type will save CU.

Recommendation:

Modify the platform_fee data type throughout the relevant files.

Developer Response:

Fixed according to our recommendations: PR #32

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification

- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

Appendix A: Merkle Tree Implementation Analysis

The following report was written during the initial assessment before the fix review.

Summary

This report examines the Merkle tree implementation used for winner verification and prize distribution in the Trepa protocol.

The implementation is functionally correct, safe and uses a sound construction algorithm with deterministic ordering. The protocol's implicit design effectively prevents any potential exploitation paths.

However, there are opportunities to align more closely with cryptographic best practices, which would provide additional defense-in-depth if the protocol architecture or leaf data structure is modified in the future.

Key Findings:

- Construction and verification algorithms are correct
- Implicit domain separation via different input lengths (40 vs 64 bytes) rather than explicit leaf/node prefixes
- Opportunities to add on-chain proof length validation for additional robustness
- No practical exploitability due to strong SHA256 properties

1. Protocol Implementation Properties

Architecture Overview

Off-chain (TypeScript):

- Location: **utils/merkleTree/createMerkleTree.ts**
- Purpose: Construct Merkle tree root from winner list
- Output: 32-byte root hash stored on-chain

On-chain (Rust):

- Location: **programs/trepa/src/utils.rs:30-56**
- Purpose: Verify winner eligibility via Merkle proof
- Input: Winner pubkey, prize amount, proof array

Data Structures

Leaf Node Construction

Location: merkleHelpers.ts:11-17, utils.rs:35-39

```typescript
// TypeScript (off-chain)
leaf_data = publicKey (32 bytes) || prize (8 bytes)
leaf_hash = SHA256(40-byte input)
```

TYPESCRIPT

```rust
// Rust (on-chain)
let mut data = [0u8; 40];
data[..32].copy_from_slice(&predictor_bytes);  // 32-byte pubkey
data[32..].copy_from_slice(&amount_bytes);     // 8-byte amount
leaf = hash(&data).to_bytes();
```

RUST

Properties:

- Fixed 40-byte input size
- Deterministic encoding
- Direct SHA256 hash (no prefix)

Internal Node Construction

Location: merkleHelpers.ts:25-30, utils.rs:43-52

```typescript
// TypeScript (off-chain)
internal_data = left_hash (32 bytes) || right_hash (32 bytes)
internal_hash = SHA256(64-byte input)
```

TYPESCRIPT

```rust
// Rust (on-chain)
let mut combined = [0u8; 64];
// Lexicographical ordering before concatenation
if computed_hash <= *node {
    combined[..32].copy_from_slice(&computed_hash);
    combined[32..].copy_from_slice(node);
} else {
    combined[..32].copy_from_slice(node);
    combined[32..].copy_from_slice(&computed_hash);
}
computed_hash = hash(&combined).to_bytes();
```

RUST

Properties:

- Fixed 64-byte input size
- Lexicographical sorting before hashing
- Direct SHA256 hash (no prefix)

Leaf nodes (40 bytes) and internal nodes (64 bytes) have different input lengths. This property provides an implicit but non-standard domain separation.

Tree Construction Algorithm

Location: createMerkleTree.ts:40-59

Step 1: Leaf Preparation

```typescript
// Sort entries by address (deterministic ordering)
const sortedEntries = entries.sort((a, b) =>
  Buffer.compare(a.address.toBuffer(), b.address.toBuffer())
);

// Hash each leaf
const leafHashes: Buffer[] = sortedEntries.map(entry => hashLeaf(entry));
```

Properties:

- Deterministic ordering via lexicographical address sorting
- Ensures same input set always produces same root

Step 2: Tree Construction

```typescript
function computeMerkleRoot(nodes: Buffer[]): Buffer {
  let currentLevel = nodes;
  while (currentLevel.length > 1) {
    const nextLevel: Buffer[] = [];
    for (let i = 0; i < currentLevel.length; i += 2) {
      if (i + 1 < currentLevel.length) {
        // Pair available - sort and hash
        const sortedPair = [currentLevel[i], currentLevel[i + 1]]
          .sort(Buffer.compare);
        nextLevel.push(hashPair(sortedPair[0], sortedPair[1]));
      } else {
        // Odd node - duplicate it
        nextLevel.push(hashPair(currentLevel[i], currentLevel[i]));
      }
    }
    currentLevel = nextLevel;
  }
  return currentLevel[0];
}
```

Properties:

- Lexicographical sibling ordering at each level
- Odd node handling: duplicate the last node

On-Chain Verification

Location: utils.rs:30-56

```rust
pub fn verify_merkle_proof(
    predictor: &Pubkey,
    amount: u64,
    proof: &[[u8; 32]]
) -> [u8; 32] {
    // 1. Compute leaf hash
    let leaf = hash(&data).to_bytes();
    let mut computed_hash = leaf;

    // 2. Traverse up the tree using proof nodes
    for node in proof.iter() {
        let mut combined = [0u8; 64];
        // Maintain lexicographical order
        if computed_hash <= *node {
            combined[..32].copy_from_slice(&computed_hash);
            combined[32..].copy_from_slice(node);
        } else {
            combined[..32].copy_from_slice(node);
            combined[32..].copy_from_slice(&computed_hash);
        }
        computed_hash = hash(&combined).to_bytes();
    }

    // 3. Return computed root for comparison
    computed_hash
}
```

Property: Lexicographical ordering matches off-chain construction

Single Winner

Location: createMerkleTree.ts:42

```typescript
while (currentLevel.length > 1) {  // Doesn't execute when length === 1
```

Property: Single-winner tree returns the leaf hash directly as the root

Determinism

The implementation provides determinism through:

Address Sorting: Same winner set always produces same leaf order

```
entries.sort((a, b) => Buffer.compare(...))
```

Lexicographical Sibling Ordering: Eliminates dependency on child insertion order and we get the same tree structure regardless of traversal order

```
[left, right].sort(Buffer.compare)
```

Consistent Encoding: Little-endian encoding for amounts for a cross-platform compatibility (Typescript - Rust)

Property: Given the same set of (address, prize) tuples, the algorithm always produces the same root hash, regardless of:

- Input order
- Platform (Typescript/Rust)

2. Defense-in-Depth Considerations

Explicit Domain Separation

The initial implementation relied on implicit domain separation (which was updated to explicit separation after the audit) through different input lengths (40 vs. 64 bytes) rather than explicit prefixes. While this approach is secure given SHA-256's properties, explicit prefixes are preferred to align with common industry practices:

Initial Implementation:

```
leaf_hash     = SHA256(pubkey || amount)          // 40-byte input
internal_hash = SHA256(left_hash || right_hash)   // 64-byte input
```

Industry Standard:

```
leaf_hash     = SHA256(0x00 || pubkey || amount)
internal_hash = SHA256(0x01 || left_hash || right_hash)
```

In other implementations where leaves and nodes would have the same input length, an attacker could theoretically attempt to find a value X such that:

```
SHA256(X) = SHA256(left || right)  // some internal node hash
```

If successful, they could submit **X** as a leaf and forge eligibility.

But the different input lengths (Leaf = 40 bytes, nodes = 64 bytes) make it practically impossible to find such a collision with the current technology.

Future changes, however, could accidentally create 64-byte leaves:

```rust
// Current: 40 bytes
leaf = pubkey(32) || amount(8)
```

... continued RUST

```
// Hypothetical future: 64 bytes
leaf = pubkey(32) || amount(8) || metadata(24)
```

With such data, both leaves and internal nodes have 64-byte inputs making the tree vulnerable to a second pre-image attack.

Recommendation

Add explicit 1-byte prefixes:

TypeScript:

TYPESCRIPT

```
export function hashLeaf(entry: MerkleLeaf): Buffer {
  const data = Buffer.concat([
    Buffer.from([0x00]),  // Leaf prefix
    entry.address.toBuffer(),
    prizeToBigIntBuffer(entry.prize),
  ]);
  return createHash('sha256').update(data).digest();
}

export function hashPair(left: Buffer, right: Buffer): Buffer {
  const data = Buffer.concat([
    Buffer.from([0x01]),  // Internal node prefix
    left,
    right,
  ]);
  return createHash('sha256').update(data).digest();
}
```

Rust:

RUST

```
pub fn verify_merkle_proof(predictor: &Pubkey, amount: u64, proof: &[[u8; 32]]) -> [u8; 32] {
    let mut data = [0u8; 41];  // 41 instead of 40
    data[0] = 0x00;            // Leaf prefix
    data[1..33].copy_from_slice(&predictor.to_bytes());
    data[33..41].copy_from_slice(&amount.to_le_bytes());
    let mut computed_hash = hash(&data).to_bytes();
    for node in proof.iter() {
        let mut combined = [0u8; 65];  // 65 instead of 64
        combined[0] = 0x01;            // Internal node prefix
        if computed_hash[..] <= node[..] {
            combined[1..33].copy_from_slice(&computed_hash);
            combined[33..65].copy_from_slice(node);
        } else {
            combined[1..33].copy_from_slice(node);
            combined[33..65].copy_from_slice(&computed_hash);
        }
        computed_hash = hash(&combined).to_bytes();
    }
    computed_hash
}
```

Proof Length Validation

Description

The **verify_merkle_proof** function currently accepts arbitrary-length proof arrays:

```rust
pub fn verify_merkle_proof(
    predictor: &Pubkey,
    amount: u64,
    proof: &[[u8; 32]]  // No length constraint
) -> [u8; 32] {
    let mut computed_hash = leaf;
    for node in proof.iter() {  // Accepts any length
        // ...
    }
    computed_hash
}
```

RUST

While the current protocol architecture prevents exploitation (due to the Predictor PDA requirement), adding depth validation would provide an additional layer of verification.

Hypothetical Substitution

Let's consider an hypothetical scenario where:

- Leaf length = node length (which is not the case currently making it impractical)
- No domain separation
- Possibility to claim on a user's behalf (which is not the case with the current protocol architecture)

The steps would be:

a. Build a modified data such as:

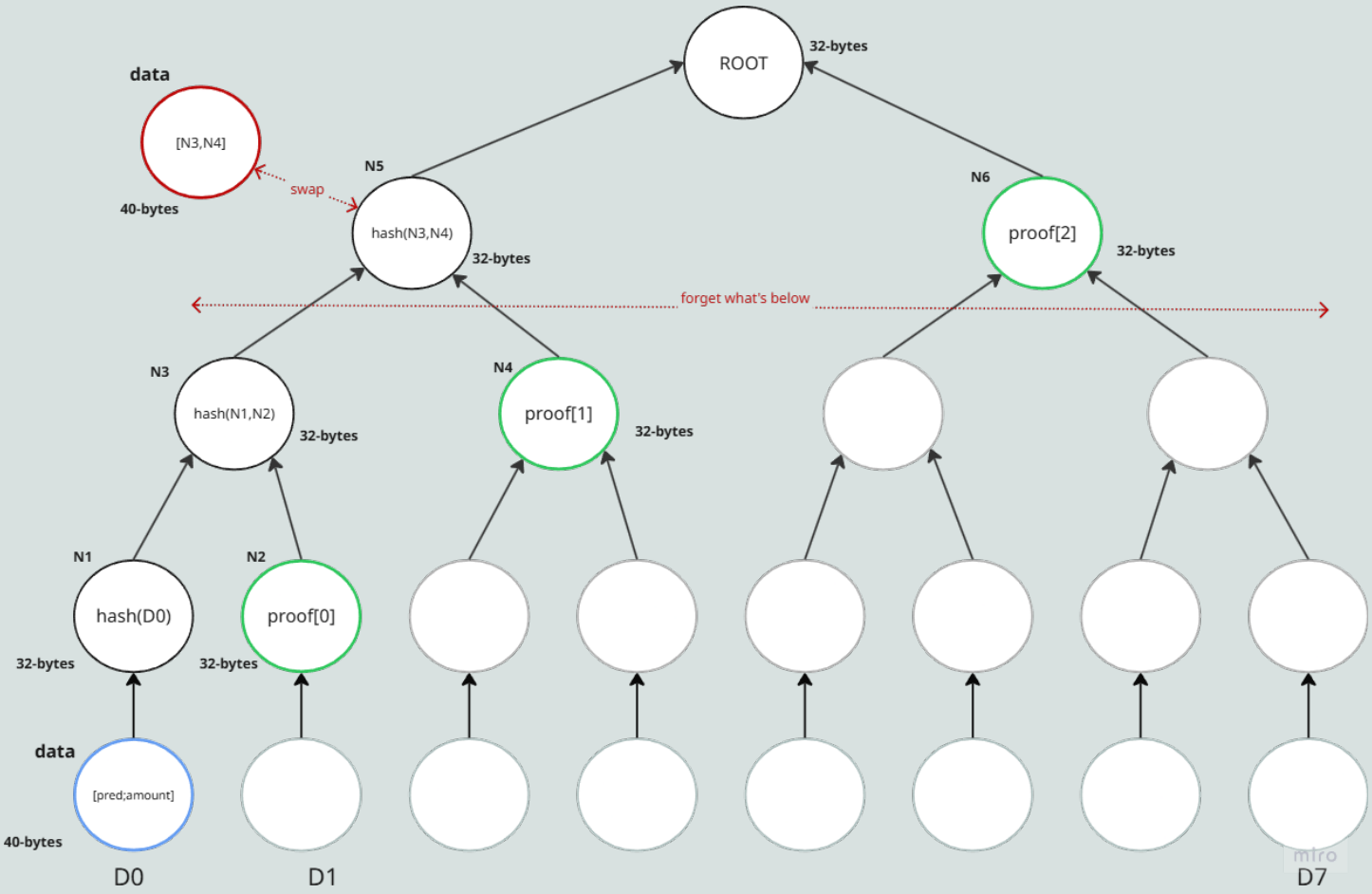
```rust
modified_data = [predictor; amount]
predictor = N3 = hash(N1,N2)
amount = N4 = proof[1]
```

RUST

b. Provide to the verifier function **modified_data** and **proof = proof[2]**

c. The verification algorithm will rebuild the tree and find a correct root, confirming the proof validity

d. **amount** will be transferred to a random pubkey described **N3**



Result: An attacker could claim a random prize amount to be transferred to a random address.

Note: This attack is not possible in the current implementation due to the Predictor PDA requirement, which ensures only the legitimate claimer can withdraw rewards. However, proof length validation would prevent this scenario if the authorization model changes.

Recommendation

Add proof length validation in the claim instruction:

```rust
// In claim_prize or similar instruction
let expected_depth = if competition.n_winners == 1 {
    0  // Single winner = empty proof
} else {
    (competition.n_winners as f64).log2().ceil() as usize
};

require!(
    proof.len() == expected_depth,
    CustomError::InvalidProofLength
);

let computed_root = verify_merkle_proof(predictor, amount, proof);
```

Single-Leaf Tree

Description

When there is exactly one winner, the algorithm returns the leaf hash directly as the root:

```typescript
while (currentLevel.length > 1) {  // Doesn't execute when length === 1
    // Tree construction logic
}
return currentLevel[0];  // Returns leaf_hash for single winner
```

Result:

```
Single winner: Alice, 100 tokens
Root = SHA256(alice_pubkey || 100)  // This is a leaf hash

Multiple winners: Alice and Bob
Root = SHA256(...)  // This is an internal node hash
```

Impact

The root changes "type" based on tree size:

- 1 winner: root is a leaf hash (40-byte input)
- 2+ winners: root is an internal hash (64-byte input)

Combined with lack of domain separation, this creates semantic ambiguity.

Recommendation

Option 1: Explicit Single-Winner Handling

```typescript
if (leafHashes.length === 1) {
  // Single winner - return leaf hash with clear documentation
  return hashPair(leafHashes[0], leafHashes[0]);  // Treat as duplicated node
}
```

Makes single-winner trees structurally identical to 2-winner trees where both winners are the same.

Post-Fix Conclusion

The Trepa Merkle tree implementation is algorithmically correct and safe.

The bottom-up construction with lexicographical ordering ensures deterministic roots, and the off-chain/on-chain implementations are properly synchronized.

However, the following enhancements would strengthen alignment with cryptographic best practices:

- Domain separation: The current approach (implicit separation via different input lengths) is secure. Explicit prefixes would provide additional robustness against future protocol modifications that might alter leaf structure.
- **Trepa implemented the explicit domain separation as part of the changes**
- Proof length validation: Adding this check would provide defense-in-depth, particularly valuable if future changes modify the authorization model or leaf structure.